

Aula 03 — Arquiteturas (parte 1)

Resumo 4 de 14 · arquivo aula03 · 21 imagens / 16 slides · Cap. 2 de van Steen & Tanenbaum

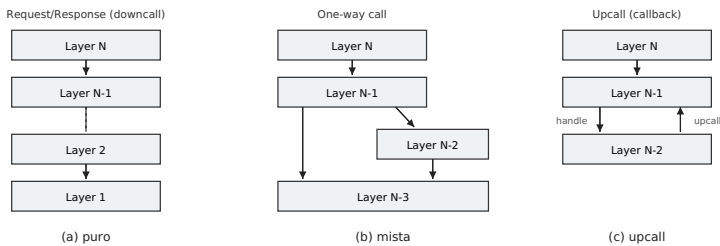
Esta aula trata dos **estilos arquiteturais**: as “formas padrão” de organizar os componentes de um sistema distribuído. Vê-se a arquitetura **em camadas**, a **estratificação de aplicações** (3 camadas), as arquiteturas **orientadas a serviço** (objetos, microsserviços, REST/SOAP) e as arquiteturas **produtor/consumidor** (acoplamento, espaço de tuplas, publicar-assinar).

1. O que é um estilo arquitetural

Um **estilo arquitetural** é definido por: (a) **componentes** substituíveis com interfaces bem definidas; (b) como eles estão **conectados**; (c) os **dados** trocados entre eles; e (d) como componentes e conectores se **configuram** num sistema. Um **conector** é o mecanismo que media comunicação, coordenação ou cooperação entre componentes — por exemplo, chamada de procedimento (remota), mensageria ou streaming.

2. Arquitetura em camadas

Componentes são organizados em camadas: uma camada só pode chamar a de baixo (e receber a resposta). Há variações na forma de chamar:

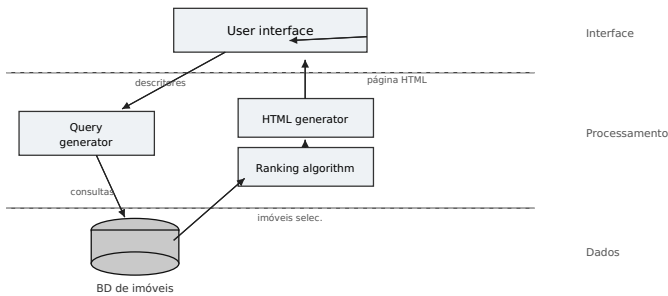


Organizações em camadas: (a) chamadas descendentes com resposta; (b) chamadas de mão única que podem pular camadas; (c) upcall — a camada de baixo chama de volta a de cima (callback).

Um exemplo clássico de camadas são os **protocolos de comunicação**, onde se distingue **protocolo** (regras entre pares), **serviço** (o que a camada oferece à de cima) e **interface** (como acessá-lo). A aula ilustra com um par de programas em **sockets** (servidor com bind/listen/accept/rcv/send e cliente com connect/send/rcv) — comunicação direta entre duas partes.

3. Estratificação de aplicações (3 camadas)

Visão tradicional que separa uma aplicação em três camadas lógicas: **interface de aplicação** (interação com usuários/sistemas externos), **processamento** (as funções da aplicação, sem dados) e **dados** (o que o cliente quer manipular). Muito comum em sistemas de informação com banco de dados. Exemplo — um buscador imobiliário:



Buscador imobiliário em 3 camadas: a interface gera descritores → o gerador de consultas acessa o BD → o ranking ordena os imóveis → o gerador de HTML monta a página de resposta.

4. Arquiteturas orientadas a serviço

- **Baseada em objetos**: componentes são **objetos** conectados por chamadas de procedimento; podem estar em máquinas diferentes (chamada pela rede). Objetos **encapsulam** dados e expõem métodos, escondendo a implementação interna.
- **Microsserviços**: evolução em que a aplicação é dividida em serviços pequenos e independentes.
- **RESTful**: vê o sistema como uma **coleção de recursos** gerenciados individualmente. Quatro princípios: (1) recursos têm **nomeação única**; (2) todos os serviços oferecem a **mesma interface**; (3) mensagens são **auto-descritivas**; (4) o serviço é **stateless** (esquece o chamador após a operação). Operações básicas:

Operação	Descrição
POST	Criar um novo recurso

GET	Recuperar o estado de um recurso
PUT	Modificar um recurso (transferindo novo estado)
DELETE	Excluir um recurso

Exemplo: **Amazon S3** — objetos (arquivos) ficam em *buckets* (diretórios), que não podem ser aninhados; cada objeto é acessado por um URI como `http://BucketName.s3.amazonaws.com/ObjectName`, usando requisições HTTP (PUT para criar, GET para listar/ler).

REST × SOAP: REST agrada por interfaces simples, mas “empurra” a complexidade para o espaço de parâmetros. Criar um bucket via **SOAP** é `bucket.create("mybucket")`; via **REST** é `PUT "https://mybucket.s3.amazonaws.com/"`. As diferenças importantes ficam na **verificação de erros** e na **semântica da operação**.

5. Arquiteturas produtor/consumidor

Aqui o foco é a **coordenação**, classificada por dois eixos: acoplamento **temporal** (produtor e consumidor precisam estar ativos ao mesmo tempo?) e **referencial** (um precisa conhecer/nomear o outro?). Isso gera quatro combinações:

	Acoplamento temporal	Desacoplamento temporal
Acoplamento referencial	Comunicação direta	Caixa de correio (mailbox)
Desacoplamento referencial	Baseado em evento	Espaço de dados compartilhado

Espaço de tuplas (estilo Linda): um exemplo de espaço de dados compartilhado, com três operações — `out(t)` adiciona a tupla `t`; `rd(t)` obtém uma *cópia* de uma tupla que casa com o modelo; `in(t)` *remove* uma tupla que casa. Chamar `out` duas vezes guarda duas cópias (o espaço é um *multiconjunto*). Tanto `in` quanto `rd` são **bloqueantes** (esperam até haver uma tupla correspondente). A aula mostra um microblog em que Bob/Alice publicam tuplas (`autor`, `tópico`, `texto`) e Chuck as lê por casamento de padrão, sem precisar conhecer o ID do publicador.

Publicar-assinar (pub/sub): como casar eventos, descritos por pares (atributo, valor)? Por **tópico** (especifica “atributo = valor”), por **conteúdo** (“atributo ∈ intervalo”) ou por **eventos compostos** (“atrib1 = v1 AND atrib2 = v2”). Atenção: assinaturas baseadas em **conteúdo** podem sofrer sérios problemas de escalabilidade (é caro casar faixas de valores em grande volume).